

Task 01 Uppercase airline names

Retrieve all airline names in uppercase.

SQL solution

```
SELECT UPPER(airline_name) AS airline_name_upper  
FROM airline;
```

Explanation

UPPER() converts each airline name to uppercase in the result. SELECT reads the data without changing the stored names.

Names already written in uppercase will look unchanged.

Task 02 Replace text in airline names

Replace every occurrence of Air with Aero in airline names.

SQL solution

```
SELECT REPLACE(airline_name, 'Air', 'Aero')  
        AS modified_airline_name  
FROM airline;
```

Explanation

REPLACE() takes the source text, the substring to find, and its replacement. The query changes only the displayed result, not the stored names.

The match is case-sensitive. Names without the exact substring Air remain unchanged.

Task 03 Flight numbers shared by two airlines

Find flight numbers associated with both airline 1 and airline 2.

SQL solution

```
SELECT flight_no
FROM flights
WHERE airline_id = 1
INTERSECT
SELECT flight_no
FROM flights
WHERE airline_id = 2;
```

Explanation

Each SELECT retrieves the flight numbers for one airline. INTERSECT returns only numbers found in both results and removes duplicates.

This interprets both as a set intersection. A single row cannot satisfy `airline_id = 1 AND airline_id = 2`. OR would include numbers from either airline.

Task 04 Airport name filters

Retrieve airports containing both Reginal and Air in their names.

SQL solution

```
SELECT airport_id, airport_name
FROM airport
WHERE airport_name LIKE '%Reginal%'
  AND airport_name LIKE '%Air%';
```

Explanation

LIKE searches for a text pattern. The percent signs allow characters before and after each substring. AND requires both patterns to match the same airport name.

Reginal is the spelling in the assignment and may be a typo for Regional. The supplied data contains neither spelling, so an empty result is expected. If the spelling is corrected by the instructor, replace Reginal with Regional.

Task 05 Format passenger birth dates

Retrieve passenger names and format birth dates as Month DD, YYYY.

SQL solution

```
SELECT first_name, last_name,  
       TO_CHAR(date_of_birth, 'FMMonth DD, YYYY')  
       AS formatted_birth_date  
FROM passengers;
```

Explanation

TO_CHAR() formats a date as text. Month gives the full month name, DD gives a two-digit day, and YYYY gives a four-digit year. FM removes padding after the month name.

In PostgreSQL, FM affects the next format element only, so DD keeps its leading zero. Example: January 05, 2004. The original date remains unchanged.

Task 06 Numbers of delayed flights

Find flight numbers that were delayed based on actual arrival.

SQL solution

```
SELECT flight_no  
FROM flights  
WHERE actual_arrival > scheduled_arrival;
```

Explanation

A flight is treated as delayed when its actual arrival is later than its scheduled arrival. The greater-than operator compares the two arrival values.

Task 6 does not explicitly state a comparison baseline; scheduled arrival is used here. The supplied columns are DATE, so this data supports day-level comparisons, not delays within the same day.

Task 07 Late arrival details

Find flights arriving later than their scheduled arrival.

SQL solution

```
SELECT flight_id, flight_no,  
       scheduled_arrival, actual_arrival  
FROM flights  
WHERE actual_arrival > scheduled_arrival;
```

Explanation

The condition identifies late arrivals. Displaying both scheduled and actual arrival makes the reason for each selected flight visible.

Tasks 6 and 7 overlap in meaning. Task 6 displays flight numbers; this task also displays the flight ID and arrival dates. Equal dates do not satisfy the condition.

Task 08 Filter airlines by country and date

Show airlines from France, Portugal or Poland created from November 1, 2023 through March 31, 2024.

SQL solution

```
SELECT *  
FROM airline  
WHERE UPPER(airline_country) IN  
      ('FRANCE', 'PORTUGAL', 'POLAND')  
      AND created_at BETWEEN DATE '2023-11-01'  
                          AND DATE '2024-03-31';
```

Explanation

IN accepts any country in the list. AND also requires the creation date to be within the range. BETWEEN includes both boundaries. UPPER() allows country values from before or after the uppercase update in Laboratory 3.

created_at is DATE in the supplied file. For a TIMESTAMP column, use created_at >= DATE '2023-11-01' AND created_at < DATE '2024-04-01' to include the whole final day.

Task 09 Three heaviest baggage items

Find the top three baggage items weighing more than 25 kg.

SQL solution

```
SELECT baggage_id, weight_in_kg, booking_id
FROM baggage
WHERE weight_in_kg > 25
ORDER BY weight_in_kg DESC, baggage_id ASC
LIMIT 3;
```

Explanation

WHERE first excludes baggage weighing 25 kg or less. ORDER BY sorts the remaining items from heaviest to lightest. LIMIT 3 returns at most three rows.

baggage_id provides a consistent order when weights are equal. The query returns three items rather than every item tied at the third weight.

Task 10 Youngest passengers

Find the full name of the youngest passenger or passengers.

SQL solution

```
SELECT CONCAT_WS(' ', first_name, last_name)
       AS full_name, date_of_birth
FROM passengers
WHERE date_of_birth = (
    SELECT MAX(date_of_birth)
    FROM passengers
);
```

Explanation

The latest birth date belongs to the youngest passenger. The subquery finds that date with MAX(); the outer query returns every passenger with that date. CONCAT_WS() joins the name parts with a space.

Using MAX() instead of MIN() is essential: a later birth date means a younger person. All ties are included.

Task 11 Cheapest booking on each platform

Find the minimum booking price for each booking platform.

SQL solution

```
SELECT booking_platform,  
       MIN(price) AS cheapest_price  
FROM booking  
GROUP BY booking_platform  
ORDER BY booking_platform;
```

Explanation

GROUP BY forms one group per booking platform. MIN(price) calculates the lowest price within each group. The result contains one row per platform.

The supplied database uses price, not ticket_price. This query returns each platform and its minimum price, not all booking details.

Task 12 Airline codes containing digits

Return airlines whose airline_code contains at least one digit.

SQL solution

```
SELECT *  
FROM airline  
WHERE airline_code ~ '[0-9]';
```

Explanation

The PostgreSQL operator ~ checks a regular expression. [0-9] matches one digit anywhere in the code, so codes containing at least one digit are selected.

The pattern has no start or end anchors. Codes containing only letters and NULL codes are not selected.

Task 13 Five newest airlines

List the five most recently created airlines.

SQL solution

```
SELECT *  
FROM airline  
ORDER BY created_at DESC NULLS LAST, airline_id ASC  
LIMIT 5;
```

Explanation

Descending date order places newer airlines first. LIMIT 5 returns at most five rows. NULLS LAST keeps unknown creation dates after known dates.

airline_id breaks ties consistently when multiple airlines have the same creation date.

Task 14 Unchecked baggage in a booking range

Return baggage check rows with booking_id from 200 to 300 inclusive and check_result different from Checked.

SQL solution

```
SELECT *  
FROM baggage_check  
WHERE booking_id BETWEEN 200 AND 300  
      AND check_result <> 'Checked';
```

Explanation

BETWEEN includes booking IDs 200 and 300. The operator <> means not equal. AND requires both the booking range and the check-result condition to hold.

A NULL check_result is not included by <>. The assignment does not request adding NULL results. This task is interpreted as referring to baggage_check.

Task 15 Updates earlier than creation

Find baggage checks updated earlier than creation within the same calendar month.

SQL solution

```
SELECT *
FROM baggage_check
WHERE update_at < created_at
      AND DATE_TRUNC('month', update_at::timestamp)
          = DATE_TRUNC('month', created_at::timestamp);
```

Explanation

The first condition finds updates dated before creation. `DATE_TRUNC()` reduces each date to the beginning of its month; equality therefore checks both year and month. `::timestamp` explicitly converts each date for the function.

Comparing only month numbers could incorrectly match different years. The field is named `update_at` in the supplied database. These rows represent inconsistent chronology.